

Programming Assignment - Arrays

RUNTIME ERRORS: If your code has a runtime error with any of my test data, you will receive a zero for the attempt. Make sure to thoroughly test your code before submitting. If you receive a zero, EVEN IF IT LOOKS LIKE YOU PASSED SOME TESTS, it is because a test after the ones you passed caused a runtime error and caused the whole testing system to crash. Since the program crashed, it means no output is generated which means the system cannot determine your grade and you receive a zero.

1. Create a file named `arrays.cc` which has the functions defined below inside.

DO NOT USE ANY CONSTANTS FROM MAIN IN YOUR FUNCTIONS. YOU MUST USE ONLY PARAMETERS OR YOU WILL FAIL THE ALL OF THE TESTS I WILL BE USING AND WILL RECEIVE A ZERO FOR A GRADE. YOU HAVE BEEN WARNED.

DO NOT CHANGE THE ARRAY ARGUMENTS THAT ARE PASSED INTO YOUR FUNCTION. Copy the data into local variables and use those local variables in your code.

YOU ARE ONLY ALLOWED TO USE `<stdio.h>` AS AN EXTERNAL LIBRARY.

2. Create a method named ***findCloseToZero*** which takes a double array and the number of items in the array as parameters. This method should return value that is closest to zero in the array. Note that it could be positive or negative value. If more than one value is possible (such as -2.0 and +2.0), use the one that comes first in the array.

```
double findCloseToZero (double myArray[], int numItems)
{
}
```

3. **YOU MUST USE POINTERS FOR THIS FUNCTION, *findFarFromZero*. If you use brackets the test will fail.**

Create a method named ***findFarFromZero*** which takes a double array and the number of items in the array as parameters. This method should return value that is farthest from zero in the array. Note that it could be positive or negative value. If more than one value is possible (such as -2.0 and +2.0), use the one that comes first in the array.

4. Create a method named ***countDivisible*** which takes an int array and the number of items in the array AND an integer divisor as parameters. This method should return the number of items in the array that are evenly divisible by the divisor.
5. Create a method named ***sumFractions*** which takes a double array and the number of items in the array and returns the sum of ONLY the fractional parts.

For example:

The following array {-2.2, -6.5, 4.25, 5.11, 2.9, -8.3, 9.4, -8.7, -4.2, 3.6}

Would sum the following: -0.2, -0.5, +0.25, +0.11, +0.9, -0.3, +0.4, -0.7, -0.2, and +0.6

And return 0.36

6. Create a method named **countAbsoluteRangeValues** which takes a double array, the number of items in the array (int), and a value to count (double) as parameters. This function should count and return (int) how many times the argument in the third parameter occurs in the array within the range of ± 0.5 . Sign should be discounted when checking if a number is in the range. For example, -2.3 should be treated as a 2.3 for range checking.

That is, if the third argument, x, is 3.1 the range of values to count would be $2.6 < |x| < 3.6$

For the array {-5.0, -3.6, **-3.1, -2.7**, -2.6, -1.2, 0.1, 1.5, 1.8, **2.7, 3.0, 3.5**, 3.6, 4.0}

```
countAbsoluteRangeValues (myArray, 14, 3.1);
```

should count all the values between -3.6 and -2.6 and between +2.6 and +3.6 and return 5.

Note this is NOT inclusive. The boundary values -2.6, -3.6, +2.6, and +3.6 should not be included in the count in the example above.

7. Use a main.cc file with a main method for testing your program before uploading. There is a main.cc on AsULearn which gives the output shown below. Test with different arrays. Make sure to test with negative elements and zeros.

DO NOT UPLOAD MAIN.CC OR ANY FILE WITH A MAIN METHOD IN IT!

The command for compiling.

```
g++ -Werror -Wall -o arrays main.cc arrays.cc
```

Run using **arrays** or **./arrays**

8. Submit your **arrays.cc** file to [Web-CAT](#) for grading.
 - a. You will lose points for late submission.
 - b. You will lose point for excessive submissions.

A SAMPLE MAIN.CC IS GIVEN ON NEXT PAGE.

```

#include <stdio.h>

//Prototypes the following six lines must
//be here or the program will not compile
double findCloseToZero(double *, int);
double findFarFromZero(double *, int);
int countDivisible(int *, int, int);
double sumFractions(double *, int);
int countAbsoluteRangeValues(double *, int, double);

void test1();

int main()
{
    test1();

    return 0;
}

void test1()
{
    //Temporary storage
    double dv = 0;
    int iv = 0;

    //Generate an array then run student functions
    double myArray[] = {2.11,3.2,0.6,1.3,2.5,5.11,6.25,6.0,5.2,4.2};
    int myIntArray[] = {2,3,0,16,20,5,6,6,5,4,8};

    //Test findCloseToZero
    dv = findCloseToZero(myArray, 10);
    printf("In function findCloseToZero your output is %f\n", dv);
    if (dv != 0.6) printf("FAIL in findCloseToZero(). Expected 0.6.\n");

    //Test findFarFromZero
    dv = findFarFromZero(myArray, 10);
    printf("In function findFarFromZero your output is %f\n", dv);
    if (dv != 6.25) printf("FAIL in findFarFromZero(). Expected 6.25.\n");

    //Test countDivisible
    iv = countDivisible(myIntArray, 11, 4);
    printf("In function countDivisible your output is %d\n", iv);
    if (iv != 5) printf("FAIL in countDivisible(). Expected 5.\n");

    //Test sumFractions
    dv = sumFractions(myArray, 10);
    //Math is tricky with doubles. Answer should be 2.41, but will actually be "very close" to 2.47
    //Assume 6 decimal places.
    //Low is 2.469999
    //High is 2.470001
    printf("In function sumFractions your output is %f\n", dv);
    if (dv < 2.469999 || dv > 2.470001) printf("FAIL in sumFractions(). Expected 2.47.\n");

    double myArray2[] = {-5.0, -3.6, -3.1, -2.7, -2.6, -1.2, 0.1, 1.5, 1.8, 2.7, 3.0, 3.5, 3.6, 4.0 };
    iv = countAbsoluteRangeValues(myArray2, 14, 3.1);
    printf("In function countAbsoluteRangeValues your output is %d\n", iv);
    if (iv != 5) printf("FAIL in countAbsoluteRangeValues(). Expected 5.\n");
}

```